Mode D Design Draft — Halide AOT + FastRPC + ISP fusion on Hexagon

Reference design for Phase 3-HX-MODE-D (Roadmap §12). Stash of the implementation artifacts (Halide generator, IDL, CMake configuration, deployment scripts) with three bug fixes applied that surfaced during the architectural review. Not active code — lives here until the Mode D agent spawns.

The end-state architecture is documented in PPT-LAT-Systems Appendix C; this document is the implementation-detail companion.

1. Bug fixes applied vs. the original proposal

Three structural issues were caught before any code landed:

1.1 HardSwish formula correction

The original `activated(x, y) = up(x, y) * swish_factor` where `swish_factor = clamp(gate_val + three, 0, six)` was NOT HardSwish. It computed `up · 6 · HardSigmoid(gate)`, which is numerically unrelated to SiLU and would have tanked PPL.

The correct HardSwish-SwiGLU is:

```
activated(x, y) = up_val * gate_val * clamp(gate_val + three, 0, six) / six
```

The missing `· gate / 6` term is what makes it a SiLU approximation rather than a HardSigmoid scaled by 6.

1.2 Gemma3 uses GeGLU, not SwiGLU

The original proposal assumed SwiGLU on every arch. Gemma3 actually uses GeGLU (`gelu_tanh(gate · x) * (up · x)`). HardSwish does not approximate GELU-tanh; it approximates SiLU. Silently substituting on Gemma3 would have failed the forward-correctness gate without pointing at the cause.

The fix: the Mode D path dispatches on `sp_arch_info.ffn_variant` at Halide generator time, emitting per-arch static archives (`ffn_skeleton_llama3.a`, `ffn_skeleton_gemma3.a`, etc.). The L1 backend links the right archive by arch_id at engine build time — no runtime branch.

For Gemma3's GeGLU, the fixed-point approximation is:

```
gelu(x) ≈ 0.5 · x · (1 + tanh_approx(0.7978 · (x + 0.044715 · x³)))

tanh_approx(y) = clamp(y, -2.5, 2.5) · (1 - y² · (a₁ + y² · (a₂ + y² · a₃)))
```

where a_1, a_2, a_3 are 5th-order least-squares coefficients fit against exact tanh on `[-2.5, 2.5]`. The actual coefficients are computed at generator time and baked into the archive.

1.3 Halide schedule fix — `compute_at(ffn_out, xi)`, not `compute_at(ffn_out, x)`

The original schedule used `up.compute_at(ffn_out, x)` and `gate.compute_at(ffn_out, x)`. This forces Halide to re-materialize the intermediate accumulators per output element rather than per vector tile, which strips out the HVX register-reuse and emits scalar `mpyih` instructions instead of vectorized `vmpa.h` patterns.

The fix: `compute_at(ffn_out, xi)` keeps the accumulation inside the inner vector loop, generating proper 128-byte HVX dot-product patterns. Verify by inspecting `objdump` of the compiled `.a`.

1.4 Float scales → int32 Q-point at session create

The original IDL had `in float* scales`. The DSP-side Halide kernel operates in int32 fixed-point; passing floats forces an emulated float→fixed cast on every layer. Real cost: ~100µs per layer at Gemma3-1B scale, gone on every decode step.

The fix: pre-convert scales to int32 Q-point (`Q_BITS=8`) at `sp_session_create` time inside the Hexagon backend's session initializer. Pass `int32* scales_q` over FastRPC. One-time conversion, zero hot-path emulated math. IDL parameter type updated accordingly.

2. Halide generator (with bug fixes applied)

`src/backends/hexagon/halide/ffn_skeleton_generator.cpp` (Mode D agent will create this fresh; do not copy from the old cohort):

```
#include "Halide.h"
using namespace Halide;

class FFNSkeletonGenerator : public Generator<FFNSkeletonGenerator> {
public:
    GeneratorParam<std::string> arch{"arch", "swiglu"};   // "swiglu" | "geglu"

    // Inputs (mapped from SVM buffers; weights f-side already Q8)
    Input<Buffer<int16_t>> residual_in{"residual_in", 2};
    Input<Buffer<int8_t>>  w_up      {"w_up",       2};
    Input<Buffer<int8_t>>  w_gate    {"w_gate",     2};
    Input<Buffer<int8_t>>  w_down    {"w_down",     2};
    Input<Buffer<int32_t>> scales_q  {"scales_q",   1};   // int32 Q8, pre-converted

    // Output
    Output<Buffer<int16_t>> ffn_out {"ffn_out", 2};

    void generate() {
        Var x("x"), y("y");
        const int Q_BITS = 8;
        const int32_t three_q = 3 << Q_BITS;
        const int32_t six_q   = 6 << Q_BITS;

        // Up and Gate projections — int32 accumulator
        RDom r(0, w_up.dim(1).extent());
        Func up("up"), gate("gate");
        up  (x, y) += cast<int32_t>(residual_in(r, y)) * cast<int32_t>(w_up  (x, r));
        gate(x, y) += cast<int32_t>(residual_in(r, y)) * cast<int32_t>(w_gate(x, r));

        // Per-arch activation — dispatched at generator time
        Func activated("activated");
        Expr up_val   = up  (x, y);
        Expr gate_val = gate(x, y);

        if (arch == "swiglu") {
```

```
            // HardSwish-SwiGLU: up · gate · clamp(gate+3, 0, 6) / 6
            // BUG FIX 1.1: include the · gate / 6 term
            Expr clamp_term = clamp(gate_val + three_q, 0, six_q);
            activated(x, y) = (up_val * gate_val * clamp_term) / (six_q << Q_BITS);
        } else if (arch == "geglu") {
            // Piecewise polynomial GeGLU per §C.4
            // BUG FIX 1.2: GELU-tanh approximation, not HardSwish
            Expr g  = gate_val;
            Expr g2 = (g * g) >> Q_BITS;
            Expr g3 = (g2 * g) >> Q_BITS;
            // Coefficients are fit at generator-init time; placeholder constants here
            const int32_t a1_q = /* fit */ 0;
            const int32_t a2_q = /* fit */ 0;
            const int32_t a3_q = /* fit */ 0;
            Expr poly = a1_q + ((g2 * (a2_q + ((g2 * a3_q) >> Q_BITS))) >> Q_BITS);
            Expr inner = (g + ((g3 * /* 0.044715_q */ 0) >> Q_BITS)) * /* 0.7978_q */ 0;
            Expr tanh_y = clamp(inner, -(int32_t)(2.5 * (1 << Q_BITS)),
                                       +(int32_t)(2.5 * (1 << Q_BITS)));
            Expr tanh_approx = tanh_y * (((int32_t)1 << Q_BITS) - ((tanh_y * tanh_y * poly
            Expr gelu_g = (g * (((int32_t)1 << Q_BITS) + tanh_approx)) >> (Q_BITS + 1);  /
            activated(x, y) = (up_val * gelu_g) >> Q_BITS;
        }

        // Down projection — int32 accumulator
        RDom r_down(0, w_down.dim(1).extent());
        Func down("down");
        down(x, y) += activated(r_down, y) * cast<int32_t>(w_down(x, r_down));

        // Scale back to int16 and add the residual
        // BUG FIX 1.4: scales_q is int32 Q-point already; no float math
        ffn_out(x, y) = residual_in(x, y)
                      + cast<int16_t>((down(x, y) * scales_q(x)) >> (2 * Q_BITS));
    }

    void schedule() {
        if (get_target().has_feature(Target::Hexagon)) {
            Var xi("xi"), yi("yi");
            ffn_out.compute_root()
                   .hexagon()
                   .tile(x, y, xi, yi, 64, 4)
                   .vectorize(xi);

            // BUG FIX 1.3: compute_at(ffn_out, xi), NOT compute_at(ffn_out, x)
            // Keeps accumulation inside the inner vector loop; emits vmpa.h not mpyih
            up  .compute_at(ffn_out, xi).vectorize(x);
            gate.compute_at(ffn_out, xi).vectorize(x);
        } else {
            // ARM NEON fallback for development on the host
            ffn_out.compute_root().vectorize(x, 8);
        }
    }
};
```

```
HALIDE_REGISTER_GENERATOR(FFNSkeletonGenerator, ffn_skeleton)
```

The GeGLU polynomial coefficients (a1_q, a2_q, a3_q, and the 0.7978 / 0.044715 / 2.5 constants in Q8) are fit by a host-side Python helper at generator construction time; the Mode D agent will publish the actual values in the SESSION-STATE closure entry.

3. IDL contract (with bug fix 1.4 applied)

`src/backends/hexagon/idl/ffn_fusion.idl`:

```
#include "remote.h"

interface ffn_fusion {
    // BUG FIX 1.4: scales is int32* (Q-point), NOT float*
    // All *Len params MUST exactly equal the rpcmem_alloc size
    //   (per feedback_fastrpc_exact_alloc — over-allocation fails AEE_EUNSUPPORTED)

    long run_ffn_skeleton(
        in   int16* residual_in, int residual_inLen,
        in   int8*  w_up,        int w_upLen,
        in   int8*  w_gate,      int w_gateLen,
        in   int8*  w_down,      int w_downLen,
        in   int32* scales_q,    int scales_qLen,
        rout int16* ffn_out,     int ffn_outLen,    // rout, not inout
        in   int32  hidden_dim,
        in   int32  seq_len
    );
};
```

4. Session lifecycle (FastRPC + SVM)

Inside sp_session_create (Hexagon backend implementation only):

```
// Allocate ION buffers sized against max_context (one-time)
size_t scratch_size = arch_info.hidden_dim * config.max_context * sizeof(int16_t);

rpcmem_init();
session->svm_residual = rpcmem_alloc(RPCMEM_HEAP_ID_SYSTEM,
                                     RPCMEM_DEFAULT_FLAGS,
                                     scratch_size);
session->svm_ffn_out  = rpcmem_alloc(RPCMEM_HEAP_ID_SYSTEM,
                                     RPCMEM_DEFAULT_FLAGS,
                                     scratch_size);
session->svm_scales_q = rpcmem_alloc(RPCMEM_HEAP_ID_SYSTEM,
                                     RPCMEM_DEFAULT_FLAGS,
                                     arch_info.hidden_dim * sizeof(int32_t));

if (!session->svm_residual || !session->svm_ffn_out || !session->svm_scales_q) {
    /* unwind, free what we got */
    return SP_ENOMEM;
}

// BUG FIX 1.4: convert f32 Frobenius scales to int32 Q-point ONCE
const int Q_BITS = 8;
```

```
for (int row = 0; row < arch_info.hidden_dim; ++row) {
    float s = model->frobenius_scales[row];
    ((int32_t*)session->svm_scales_q)[row] = (int32_t)(s * (1 << Q_BITS));
}
```

Inside sp_session_destroy:

```
rpcmem_free(session->svm_residual);
rpcmem_free(session->svm_ffn_out);
rpcmem_free(session->svm_scales_q);
rpcmem_deinit();
```

Inside sp_prefill_chunk / sp_decode_step — zero rpcmem_alloc. Just memcpy (or better, decode directly into) the pre-allocated ION pointers and dispatch.

5. CMake split-architecture build

src/backends/hexagon/CMakeLists.txt:

```
# ------------------------------------------------------------------------------
# Phase 3-HX-MODE-D: Halide AOT + FastRPC build
# ------------------------------------------------------------------------------

set(HEXAGON_SDK_ROOT $ENV{HEXAGON_SDK_ROOT})
# qaic.exe lives in WinNT/ subdir (per reference_hexagon_build_recipe)
set(QAIC_EXE        ${HEXAGON_SDK_ROOT}/ipc/fastrpc/qaic/WinNT/qaic.exe)
set(HEXAGON_CLANG   ${HEXAGON_SDK_ROOT}/tools/HEXAGON_Tools/8.5.08/Tools/bin/hexagon-clang

# Halide host-side generator
add_executable(generate_ffn_skeleton
    halide/ffn_skeleton_generator.cpp
)
target_link_libraries(generate_ffn_skeleton PRIVATE Halide::Halide)

# Per-arch Halide AOT outputs
foreach(arch_variant IN ITEMS llama3 qwen3 gemma3 deepseek_v4)
    if(arch_variant STREQUAL "gemma3")
        set(halide_arch "geglu")
    else()
        set(halide_arch "swiglu")
    endif()

    add_custom_command(
        OUTPUT  ${CMAKE_CURRENT_BINARY_DIR}/ffn_skeleton_${arch_variant}.a
                ${CMAKE_CURRENT_BINARY_DIR}/ffn_skeleton_${arch_variant}.h
        COMMAND generate_ffn_skeleton
                -g ffn_skeleton
                -e static_library,h
                -o ${CMAKE_CURRENT_BINARY_DIR}
                -p ${CMAKE_CURRENT_BINARY_DIR}/ffn_skeleton_${arch_variant}
                target=hexagon-v69-no_asserts arch=${halide_arch}
        DEPENDS generate_ffn_skeleton
        COMMENT "Halide AOT compile: ffn_skeleton (${arch_variant} → ${halide_arch})"
    )
```

```
endforeach()

# QAIC IDL → stub + skel
add_custom_command(
    OUTPUT  ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion_stub.c
            ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion_skel.c
            ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion.h
    COMMAND ${QAIC_EXE} -mdll -I ${HEXAGON_SDK_ROOT}/inc/stddef
            ${CMAKE_CURRENT_SOURCE_DIR}/idl/ffn_fusion.idl
    DEPENDS ${CMAKE_CURRENT_SOURCE_DIR}/idl/ffn_fusion.idl
    WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}
)

# DSP-side compute target — hexagon-clang, NOT NDK
add_custom_command(
    OUTPUT  ${CMAKE_CURRENT_BINARY_DIR}/libffn_fusion_skel.so
    COMMAND ${HEXAGON_CLANG} -O3 -mv69 -G0 -shared -fPIC
            -I${HEXAGON_SDK_ROOT}/inc/stddef
            -I${HEXAGON_SDK_ROOT}/inc/fastrpc
            -I${CMAKE_CURRENT_BINARY_DIR}
            ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion_skel.c
            ${CMAKE_CURRENT_SOURCE_DIR}/ffn_fusion_imp.c
            ${CMAKE_CURRENT_BINARY_DIR}/ffn_skeleton_llama3.a   # one or more per-arch arc
            -o ${CMAKE_CURRENT_BINARY_DIR}/libffn_fusion_skel.so
            -lhexagon
    DEPENDS ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion_skel.c
            ${CMAKE_CURRENT_SOURCE_DIR}/ffn_fusion_imp.c
            ${CMAKE_CURRENT_BINARY_DIR}/ffn_skeleton_llama3.a
)

add_custom_target(dsp_skel ALL DEPENDS ${CMAKE_CURRENT_BINARY_DIR}/libffn_fusion_skel.so)

# ARM-side L1 engine — Android NDK
target_sources(sp_engine_hexagon PRIVATE
    ${CMAKE_CURRENT_BINARY_DIR}/ffn_fusion_stub.c
    forward_hexagon.c
)
target_include_directories(sp_engine_hexagon PRIVATE
    ${CMAKE_CURRENT_BINARY_DIR}
    ${HEXAGON_SDK_ROOT}/inc/stddef
    ${HEXAGON_SDK_ROOT}/inc/fastrpc
)
```

6. Deployment script (`scripts/deploy/deploy-s22u.bat`)

Fresh implementation inside the lattice cohort. The semantic patterns (ADSP_LIBRARY_PATH trailing semicolon, freethedsp opt-in, SP_ENGINE_NTT_ATTN=1) carry forward from the old cohort per anti-contamination rules; the code is rewritten.

```
@echo off
setlocal enabledelayedexpansion

set DEVICE_TARGET_DIR=/data/local/tmp/shannon-prime
```

```
set LOCAL_BUILD_ARM=build-android-arm64
set LOCAL_BUILD_DSP=build-hexagon

adb devices | findstr /C:"device" >nul
if errorlevel 1 (
   echo [ERROR] No Android device detected. Connect S22U + enable USB Debugging.
    exit /b 1
)

adb shell "mkdir -p !DEVICE_TARGET_DIR! !DEVICE_TARGET_DIR!/dsp"

adb push !LOCAL_BUILD_ARM!\bin\sp_engine_runner !DEVICE_TARGET_DIR!/
adb push !LOCAL_BUILD_ARM!\lib\libshannonprime.so !DEVICE_TARGET_DIR!/
adb push !LOCAL_BUILD_DSP!\libffn_fusion_skel.so !DEVICE_TARGET_DIR!/dsp/

set MODEL_FILE=D:\Files\Models\Mine\gemma-3-1b-it\gemma-3-1b-it-f16.sp-model
set    TOKENIZER_FILE=D:\Files\Models\Mine\gemma-3-1b-it\gemma-3-1b-it-f16.sp-
tokenizer
adb push !MODEL_FILE!     !DEVICE_TARGET_DIR!/model.sp-model
adb push !TOKENIZER_FILE! !DEVICE_TARGET_DIR!/tokenizer.sp-tokenizer

adb shell "chmod +x !DEVICE_TARGET_DIR!/sp_engine_runner"

:: ADSP_LIBRARY_PATH MUST end with a trailing semicolon — Qualcomm loader
:: tokenizes the path string and silently fails parse without it
adb shell "export LD_LIBRARY_PATH=!DEVICE_TARGET_DIR!:\$LD_LIBRARY_PATH && ^
           export ADSP_LIBRARY_PATH=\"!DEVICE_TARGET_DIR!/dsp;\" && ^
           export SP_FREETHEDSP=1 && ^
           export SP_HX_MODE=D && ^
           export SP_ENGINE_NTT_ATTN=1 && ^
           !DEVICE_TARGET_DIR!/sp_engine_runner ^
                --model !DEVICE_TARGET_DIR!/model.sp-model ^
                --tokenizer !DEVICE_TARGET_DIR!/tokenizer.sp-tokenizer ^
                --prompt \"The capital of France is\""

endlocal
```

7. Verification checklist (what the Mode D agent must prove before closure)

In order, each step ungating the next:

1. Halide generator builds on host; emits all 4 per-arch `.a` archives.
2. `objdump -d ffn_skeleton_llama3.a` shows `vmpa.h` instructions (not `mpyih`).
3. `qaic.exe` emits `ffn_fusion_stub.c` and `ffn_fusion_skel.c` without errors (verify Git `sh.exe` in PATH per `reference_hexagon_build_recipe`).
4. `hexagon-clang` links `libffn_fusion_skel.so`.
5. NDK builds `libshannonprime.so` linking the stub.
6. `deploy-s22u.bat` lands both `.so`s on the device.
7. ADSP_LIBRARY_PATH trailing semicolon verified by inspection.
8. SP_HX_MODE=D engages — `sp_status` returns SP_OK.
9. Per-arch SwiGLU/GeGLU parity gates (E_HXD_3) green.
10. Three-way parallel correctness (E_HXD_4) green.
11. T_FRO_4 split gate on Mode D (E_HXD_6) green.

12. Sustained 5-min decode without SP_EHX_THERMAL_TRIP.
13. Tag `lat-phase-3-hx-mode-d-closed` and write SESSION-STATE.

Status. Reference design only. Not active code. Activates when Phase 3-HX-MODE-D agent spawns (blocked by `lat-phase-3-hx-mode-c-closed`).